

Lesson 4: Intro to Data Engineering

Python Essentials — Code the Dream

Game Plan for Today

Section	Time	What We'll Do
Warm-Up	5 min	Guess the output — lists vs. tables
I Do	10–15 min	Pandas basics: Series, DataFrames, loading data
We Do	15–20 min	Build and inspect a DataFrame together
You Do	5–10 min	Data selection challenge
Wrap-Up	5 min	Assignment preview & Q&A

Warm-Up (5 min)

From Dicts to Data

```
data = {  
    "Name": ["Alice", "Bob", "Charlie"],  
    "Age": [25, 30, 35]  
}
```

Quick questions:

1. How would you get Bob's age using plain Python?
2. What if this dict had 10,000 entries — would that approach still work?

Share your answer in the chat.

I Do: What is Pandas?

Pandas is Python's go-to library for working with tabular data.

```
import pandas as pd
```

Why Pandas?

Need	Pandas Solution
Load a CSV file	<code>pd.read_csv()</code>
Filter rows	<code>df[df["Age"] > 25]</code>
Get summary stats	<code>df.describe()</code>
Handle missing data	<code>df.fillna()</code>

Think of Pandas as **Excel + Python** — the power of spreadsheets with the flexibility of code.

I Do: Series — A Single Column

A **Series** is a labeled one-dimensional array:

```
import pandas as pd

ages = pd.Series([25, 30, 35], index=["Alice", "Bob", "Charlie"])
print(ages)
```

```
Alice      25
Bob        30
Charlie    35
dtype: int64
```

I Do: Series — A Single Column (Cont.)

Key ideas:

- Each value has a **label** (the index)
- Access by label: `ages["Bob"]` → `30`
- Access by position: `ages.iloc[1]` → `30`

I Do: DataFrame — The Full Table

A **DataFrame** is a 2D table — like a dict of Series:

```
data = {  
    "Name": ["Alice", "Bob", "Charlie"],  
    "Age": [25, 30, 35],  
    "City": ["New York", "Los Angeles", "Chicago"]  
}  
df = pd.DataFrame(data)  
print(df)
```

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

Each column is a Series. Each row has an index label.

I Do: Loading Data from Files

CSV — the most common format:

```
df = pd.read_csv("employees.csv")
```

JSON — common for web data:

```
df = pd.read_json("data.json")
```

From a dictionary (already saw this):

```
df = pd.DataFrame(my_dict)
```


I Do: Loading Data from Files (Cont.)

Format	Method
CSV	<code>pd.read_csv()</code>
JSON	<code>pd.read_json()</code>
Dictionary	<code>pd.DataFrame()</code>

I Do: Inspecting Your Data

Once data is loaded, **always inspect it first:**

```
df.head(3)      # First 3 rows
df.tail(2)      # Last 2 rows
df.info()       # Column types, null counts, memory
df.shape        # (rows, columns) as a tuple
```

info() output tells you:

- How many rows you have
- Which columns have missing values
- What data type each column is

Make `df.info()` your first move after loading any dataset.

I Do: Selecting Data

Select a column:

```
df["Age"]           # Returns a Series  
df[["Name", "City"]] # Returns a DataFrame
```

Select by position:

```
df.iloc[1]          # Second row (all columns)  
df.iloc[1, 2]        # Second row, third column
```

Select by label:

```
df.loc[1, "Name"]    # Row with index 1, Name column
```

I Do: Selecting Data (Cont.)

Filter rows:

```
df[df["Age"] > 25] # All rows where Age > 25
```

I Do: Adding and Modifying Columns

Add a new column:

```
df["Salary"] = [70000, 80000, 90000]
```

Modify an existing column:

```
df["Age"] = df["Age"] + 1 # Everyone ages a year
```

Operations apply to the whole column at once — no loop needed!

```
# This Pandas one-liner replaces a for loop:  
df["Senior"] = df["Age"] > 30
```

I Do: Saving Data

Save to CSV:

```
df.to_csv("employees.csv", index=False)
```

- `index=False` prevents Pandas from writing the row numbers as a column
- The output is a plain text file you can open in Excel or any editor

Why `index=False`?

Without it, you get an extra unnamed column when you reload:

```
,Name,Age,City  
0,Alice,25,New York
```

I Do: Combining DataFrames

`pd.concat()` stacks DataFrames vertically:

```
df1 = pd.DataFrame({"Name": ["Alice"], "Age": [25]})
df2 = pd.DataFrame({"Name": ["Bob"], "Age": [30]})

combined = pd.concat([df1, df2], ignore_index=True)
print(combined)
```

	Name	Age
0	Alice	25
1	Bob	30

- `ignore_index=True` resets the index to 0, 1, 2...
- Both DataFrames must have the **same column names**

We Do: Build a DataFrame from Scratch

Let's recreate what the assignment asks — step by step.

Step 1: Create a DataFrame from a dictionary:

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [25, 30, 35],
    "City": ["New York", "Los Angeles", "Chicago"]
}
df = pd.DataFrame(data)
print(df)
```

What do you expect the output to look like?

We Do: Add a Column and Modify

Step 2: Add a Salary column:

```
df_with_salary = df.copy()
df_with_salary["Salary"] = [70000, 80000, 90000]
print(df_with_salary)
```

Step 3: Increment Age by 1:

```
df_older = df_with_salary.copy()
df_older["Age"] = df_older["Age"] + 1
print(df_older)
```

Why do we use `.copy()` ?

Without it, changes to `df_older` could affect the original `df`. Copies keep your data safe.

We Do: Save and Reload

Step 4: Save to CSV:

```
df Older.to_csv("employees.csv", index=False)
```

Step 5: Load it back:

```
reloaded = pd.read_csv("employees.csv")  
print(reloaded)
```

Quick check: Does the reloaded DataFrame look the same as the original?

The round-trip (dict → DataFrame → CSV → DataFrame) is one of the most common data engineering patterns.

We Do: Inspect the Data

Step 6: Use inspection methods:

```
print(reloaded.head(2))  
print(reloaded.tail(1))  
print(reloaded.shape)  
reloaded.info()
```

Discussion: What does each method tell you?

Method	What You Learn
head(n)	First n rows — quick sanity check
tail(n)	Last n rows — check for trailing junk
shape	Total (rows, columns) count
info()	Types, nulls, memory — the full picture

You Do: Data Selection Challenge (5 min)

Given this DataFrame:

```
df = pd.DataFrame({  
    "Product": ["Laptop", "Phone", "Tablet", "Monitor"],  
    "Price": [1200, 800, 450, 350],  
    "Stock": [15, 42, 30, 8]  
})
```

Write code to:

1. Select just the `Product` column
2. Get the row for "Tablet" (hint: it's at index 2)
3. Filter for products with `Price > 500`

Bonus: What's the average price? (`df["Price"].mean()`)

Assignment Preview

This week's assignment is in `assignment4.py` — all tasks build on each other:

Task	What You'll Do
1	Create a DataFrame, add columns, modify, save to CSV
2	Load CSV and JSON, combine DataFrames
3	Inspect with <code>head()</code> , <code>tail()</code> , <code>shape</code> , <code>info()</code>
4	Clean dirty data: duplicates, types, missing values

Run tests with: `pytest -v -x assignment4-test.py`

Assignment Tips

Task 1: Remember `df.copy()` when making modified versions — the tests check that originals are unchanged.

Task 2 (JSON): You need to *create* `additional_employees.json` yourself. Use a list of dicts format:

```
[{"Name": "Eve", "Age": 28, ...}, ...]
```

Task 4 (cleaning): Take it step by step — each sub-task has its own test. Use `pd.to_numeric(..., errors="coerce")` for tricky columns.

Wrap-Up

Today we covered:

- Pandas Series and DataFrames
- Loading data from CSV, JSON, and dictionaries
- Inspecting data with `head()`, `tail()`, `info()`, `shape`
- Selecting, filtering, adding, and modifying columns
- Saving and reloading data

Quick check: On a scale of 1–5, how comfortable do you feel creating a DataFrame from a dictionary?

Getting Help This Week

- Start with **Tasks 1–3** before tackling Task 4 (data cleaning)
- Use the **Python interactive shell** to test Pandas commands
- Post questions in **#discussion** on Slack
- Book a **1:1 mentor session** if DataFrames feel confusing
- **Pandas docs are your friend:** pandas.pydata.org/docs

Next week: Data Cleaning and Validation — making messy data analysis-ready!

Welcome to Data Engineering!

The real fun begins — let's wrangle some data.

See you next session!